

ML System Design Core Concepts

Feature Engineering 🎧

How to structure the feature discussion in an ML system design interview without falling into the tarpit. Categorization, encoding, normalization, online/offline parity, and the pitfalls that disqualify candidates.

The feature discussion is the single biggest tarpit in the ML system design interview. We see this in our mock interviews over and over: candidates either dump 30 features in a stream-of-consciousness and run out the clock, or they freeze because they don't know the domain well enough to confidently propose specifics. Both fail the interview for the same reason — there's no structure underneath the words.

This piece is about the structure. If you have a way to organize the feature conversation, you can talk about features for as long as the interviewer wants and stop without looking incomplete when they want to move on. Without it, you're at the mercy of whatever associative chain your brain decides to follow.

How Features Have Evolved

Most interview prep treats feature engineering as the act of hand-crafting numeric features for a logistic regression or a GBDT. `engagement_count_per_view...`, that flavor. Fifteen years ago that was the job ... and it mostly isn't anymore.

Modern production ML systems lean on transformers, DLRM-style architectures, and multimodal foundation models that consume raw text, raw images, and sequences of events directly. The model figures out the interactions. Hand-tuned numeric features still exist (light rankers and tree models love them, fraud and trust-and-safety teams still ship dozens), but they're a minority of what actually goes into a top-of-funnel ranking system at YouTube, LinkedIn, Meta, or Pinterest. The Generative Recommender that LinkedIn recently published attends over 1000+ historical interactions per user as raw tokens. Meta replaced its DLRM-style ads ranker with a transformer-based sequence model in 2024, where event-based features come in as embeddings instead of hand-engineered counts. Pinterest's TransActV2 feeds up to 16,000 lifelong user actions through a self-attention transformer. There's almost no traditional "feature engineering" in any of those paths.

So why is the feature discussion still 30% of the ML system design interview? Because *what data sources you put in front of the model* is still a design decision the model can't make for itself. A transformer can learn that user dwell time interacts with creator reputation in some non-linear way, but only if you feed it both signals. Most "the model isn't learning what we want" outcomes in production come down to a missing source, a stale source, or a source computed inconsistently between training and serving. None of those problems get easier when you swap an MLP for a transformer.

The modern feature discussion is less about *engineering* features and more about identifying signal sources, choosing how to expose them, keeping them fresh, and keeping training and serving in sync. That's what we'll get into here.

The Sources of Signal Real Systems Use

I said earlier that this discussion is a tarpit and that engineers get stuck here in their presentation. This is universally true, it's just very hard to keep track of time while you're engaged in a very creative aspect of solution formulation. The trick to staying out of the tarpit is to enumerate signal *sources*, not low-level features. Real systems pull from a small handful of sources, and every problem you'll do touches a subset of them. Sketching the sources first, then populating each with a few representative signals, gives you a structure the interviewer can follow and lets you stop at any time when they want to move on.

The sources are remarkably consistent across problems. The same five show up in our bot detection, harmful content, and video recommendations breakdowns, with different ones doing the heavy lifting depending on the problem.

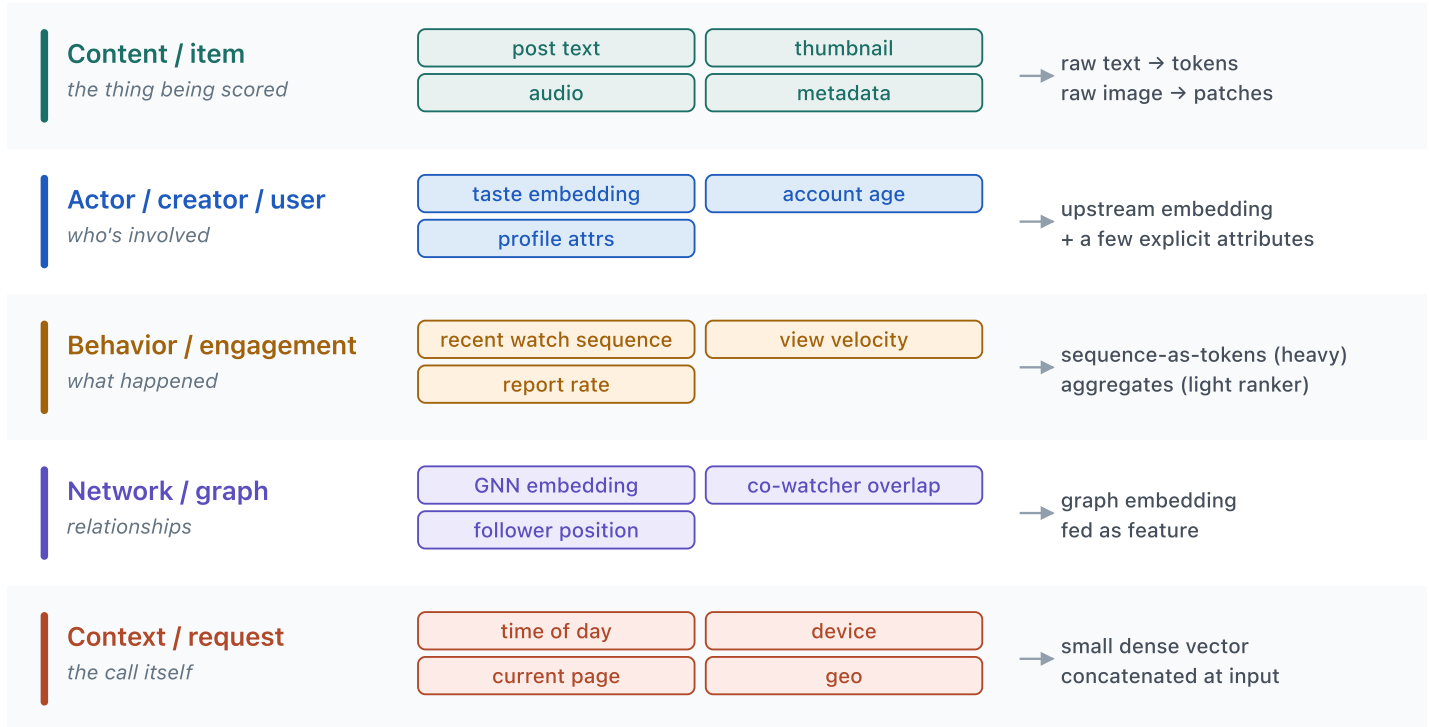
A lot of new engineers stop at content/item signals and call it a day. Knowing about all of them will help you power through. Let's go through them one by one.

Where real systems get signal

Five sources, repeated across almost every ML system design problem you'll see

EXAMPLE SIGNALS

MODEL INTAKE



Content / item signals

These are the thing being scored. The post text and attached image for harmful content. The video itself (title, thumbnail, audio, transcript) for recommendations. The transaction details for fraud. The candidate document for ranking.

In an old-school setup, you'd hand-extract n-gram counts, image color histograms, file size buckets. In a modern setup, you tokenize the text and let a transformer encode it, you patch-encode the image, you feed the multimodal pair into a single backbone. DoorDash's [DashCLIP](#) is a 2025 example: a [CLIP](#)-style contrastive model trained on 32M query-product pairs that aligns product images, product text, and search queries in a shared embedding space, then ships those embeddings into the ranker as features. Your job is to *expose* the raw content and create a model that will learn to consume it.



Content tends to be the highest-signal source for problems where the answer is fundamentally "what does this thing look like." Harmful content classification is dominated

by content. Recommendation often isn't — the content tells you surprisingly little about whether someone will engage, which is why behavioral signals matter more there.

Actor / creator / user signals

This is who's involved. The viewer for recs. The author for harmful content. The account holder for fraud. The candidate user for "people you may know."

Modern systems frequently represent actors with a long-term embedding learned by an upstream model. The user's tastes or the creator's reputation are captured as a vector. You feed the embedding directly. Layered on top are a handful of explicit attributes the embedding is bad at expressing: account age, sign-up country, language preference, verification status, recent flags. The combination of slow-but-rich embedding plus fast-but-noisy explicit attributes is the dominant production pattern, and the harmful content breakdown walks through this exact split.

A common shortcut in interviews is to just assume the upstream embedding exists. Big companies have user/content embeddings shared across teams, and most of the time you can say "I'd pick up the existing user embedding from some company-level representation" rather than re-deriving one in the moment. Interviewers know this is how it actually works.

Behavioral signals

Next are the behavioral signals, which frequently capture what happened like the user's last 100 watches, the video's view-velocity over the last hour, the account's posting cadence, or the post's negative-reaction rate.

This is the source that has changed most because of modern models. Where ten years ago you'd compute `clicks_in_last_hour` and feed it as a scalar, you're now just as likely to feed the raw sequence of `(item_id, action, timestamp)` tuples into a transformer and let it learn the temporal structure. Now that said, aggregates haven't gone away. Light rankers in the wide funnel still need cheap features, and tree models in fraud and trust-and-safety still consume them. But the heavy ranker or core model increasingly eats sequences directly.

Behavioral signals are also where most of the engineering pain lives. They drift, they leak, they get computed inconsistently between training and serving. We'll come back to all three in the

pitfalls section.

Network / graph signals

These are a little less common, but they're still important. Things like relationships, like who follows whom, which accounts transact with each other, which videos get co-watched in the same session, or whether two accounts share IPs or device fingerprints.

The dominant way to consume graph signal is as a learned embedding from a Graph Neural Network. You won't always hand-engineer "average follower count of the user's neighbors" anymore — the GNN learns that and a hundred other graph features automatically, and you just feed the resulting embedding as an input. Hand-crafted graph features (clustering coefficient, follower-to-following ratio) survive in adversarial domains where you specifically want interpretable signals you can audit, but the trend is toward learned representations.

This bucket is heavy in adversarial domains (bot detection, fraud) and social domains (PYMK, feed). It's mostly empty for product-classification problems (much to the annoyance of graph-modeling enthusiasts), where a single hop at most is enough to get the job done.

Context / request signals

Our last source is the the call itself. Aspects of it like time of day, day of week, device, current page, weather, or recent search queries. These are cheap to compute, almost always real-time, and surprisingly predictive — a recommendation request from a phone in bed at 11pm can probably be scored differently from a desktop at 9am and that's quite useful!

Context is the easiest bucket to fill and the easiest one to underweight. Most candidates forget it entirely. Mentioning it explicitly is a cheap way to signal that you've thought about what makes a request unique.

How to Cover Sources

While you don't need to write out all of these sources and fill them out like a checklist, it's helpful to have them in the back of your mind as you discuss the problem. Many candidates will miss key aspects of the problem simply because they haven't been exhaustive. And it's not

uncommon for a particular source to have a large ramification on how the solution is structured.

As a quick example, different signals within the same bucket move at very different rates. A creator's content embedding (slow-moving, daily refresh) and the same creator's last hour of report rate (fast-moving, streaming) are both "actor signals," but the infrastructure to serve them is completely different.

Ideally, this is coming out as you're discussing! "The title text is static, computed at upload. The view velocity is fast-moving, behind a streaming aggregator. The current scroll depth is real-time at request." The interviewer hears you thinking about serving cost and practical implementation details. Each step down the ladder (static, daily batch, streaming, request-time) costs more infrastructure than the one above it, and not every signal deserves to live at the bottom.



A senior candidate proposes a fast-moving signal and immediately asks "is this worth a streaming pipeline?" rather than handwaving "we'd compute this in real-time." Your interviewer is watching your brain move through the same steps they did when they first saw this problem.

Getting Signal in Front of a Modern Model

Once you've identified the sources, you have to actually feed them to the model. The shape of each source dictates the encoding, and the encoding question is where you graduate from a fuzzy idea "this would be cool" to something that can actually be implemented. It's quite common for candidates to get caught offguard in the "middle" here.

Before we go deep, here's the quick decision table. Each row is the dominant encoding for that shape of input today, with a paper that nails down the technique and a real production system you can read up on.

Feature shape	Modern approach	Key paper	Production example
Raw text	Tokenize → transformer encoder,	BERT (Devlin et al. 2018)	Meta's transformer ads ranker (2024)

Feature shape	Modern approach	Key paper	Production example
	jointly trained or fine-tuned		
Raw images	Patch-encode through a vision transformer	<u>ViT (Dosovitskiy et al. 2020)</u>	<u>DoorDash DashCLIP (2025)</u>
Text + image together	Contrastive alignment in a shared embedding space	<u>CLIP (Radford et al. 2021)</u>	<u>DoorDash DashCLIP (2025)</u>
Sparse categoricals (IDs, enums)	Learned embedding table, looked up by ID	<u>Wide & Deep (Cheng et al. 2016)</u>	<u>YouTube recommendations DNN (Covington et al. 2016)</u>
Very high-cardinality IDs	Hashing trick into a fixed bucket count, then embed	<u>Feature Hashing (Weinberger et al. 2009)</u>	<u>Meta DLRM (Naumov et al. 2019)</u>
Event sequences, light ranker	Mean / recency-weighted aggregation of action embeddings	<u>DIN (Zhou et al. 2017)</u>	<u>Alibaba display advertising</u>
Event sequences, heavy ranker	Sequence-as-tokens fed through self-attention	<u>Behavior Sequence Transformer (Chen et al. 2019)</u>	<u>Pinterest TransActV2 (16k lifelong actions)</u>
Power-law numeric scalars	Log-scale, or bucket-then-embed	<u>Practical Lessons at Facebook (He et al. 2014)</u>	<u>Stripe Radar (1,000+ signals per txn)</u>
Graph / network relationships	Train a GNN, import the embedding as a feature	<u>PinSage (Ying et al. 2018)</u>	<u>Pinterest PinSage in production</u>

Now let's go through each row.

Raw text and images

For images and text the pattern is solidified. For text, you'll tokenize the text, run it through an encoder. For images, you'll patch-encode the image (a la vision transformers), run it through a

vision backbone. Either you train the encoder jointly with the rest of the model (common for top-of-funnel ranking) or you start from a pre-trained backbone and either fine-tune or freeze it (common when content matters but training data is limited).

Almost no manual feature engineering happens here in modern systems. You don't extract bag-of-words counts. You don't compute color histograms. You expose the raw content and let the model do the work. If you find yourself proposing twenty hand-crafted text features, there are better ways!

Sparse categoricals (IDs, enums)

Item IDs, user IDs, category enums, hashtags. The main approach is to use an **embedding table** — one learned vector per category, looked up by ID. Basically, we're going to turn the category into a vector that the model can learn from. See the [Embeddings core concept](#) for the full treatment. These embeddings are either learned entirely by training your model or learned separately on related data and then used as an input to your model.



As an aside: many features have too many levels to be practical, and while modern models are often large, we don't want to waste parameters unnecessarily. A user-ID embedding table with 1B users at 64 dimensions is roughly 256GB of parameters before you've trained anything else, and most of those rows correspond to users who showed up twice and gave you almost no signal. Paying full price for the long tail is a bad trade.

The **hashing trick** is the standard workaround. Pick a fixed bucket count (say 10M instead of 1B), hash each ID into that range, and let multiple IDs share a row. Heavy users dominate the gradient updates on their bucket, so they still get clean signal. Long-tail users share buckets and drift toward something close to an "average user" embedding, which is roughly what you'd want for someone you've barely seen anyway. The model figures out the rest.

Sequences

Recent watches, recent clicks, recent transactions, recent posts. Sources that are themselves *lists*.

This is where the modern shift is most visible. Two ways to consume them:

- **Aggregation** — the mean of the embeddings of the last 10 watches, possibly recency-weighted. Cheap, loses ordering, works fine for a light ranker like the early stages of a recommendation system churning through millions of candidates.
- **Sequence-as-tokens** — feed the raw sequence of (item, action, timestamp) tuples into a transformer that attends over them. The model learns the temporal structure itself. This is the heavy-ranker move, and it's how DLRM, the LinkedIn Generative Recommender, and similar systems work in production today.

A senior candidate is going to know how to use both: aggregations for the wide funnel (where we need to bias toward our compute constraints), full sequences for the narrow one (where we're trying to maximize performance).

Numeric scalars

Numeric scalars are the one place where old-school feature engineering still earns its keep. A handful of transforms cover the bulk of what production systems do, and the engineering judgment here is what separates good candidates from average ones.

- **Log-scaling** is the default for power-law distributions (views, followers, dollar amounts). Without it the gap between 100k and 1M views dwarfs everything else and your model overfits to a handful of mega-popular items. If you've got something exponential, use it!
- **Bucketing and embedding** is an approach where you discretize into 10–100 buckets and embed each bucket so the numeric enters the model as a learned vector. This is most useful when the relationship between value and label isn't monotonic.
- **Standardization** (zero mean, unit variance) matters for linear models, less for trees and embeddings; whatever you pick, the parameters have to come from training data and be applied identically at serve time.

Straightforward so far. More interesting stuff happens when the scalar is a *rate*. Consider three videos: A has 12,000 views and 240 negatives, B has 8 views and 4 negatives, C has 9,000 views and 1,800 negatives. Raw counts say A is worse than C (wrong). The naive ratio **negatives/views** says B at 50% is the worst (also wrong — B has 8 views, the model shouldn't be confident about anything).

A better approach? **Bayesian smoothing**:

$$\text{smoothed} = (\text{negatives} + \alpha \cdot C) / (\text{views} + C)$$



α is your prior belief about the rate (say 2%) and C is a pseudo-count controlling how many real views it takes to overcome the prior (say 50). B's smoothed rate drops to ~8.6%, C stays close to its observed 20%. Sometimes called Wilson smoothing or additive smoothing, this is where we regress small-denominator estimates toward a global prior, and let the data overcome the prior as the denominator grows.

The other half of rate features is *recency*. Old events shouldn't count the same as new ones, but a hard cutoff ("last 24 hours") is brittle and throws away signal at the boundary. Two production techniques, often used together:

- **Exponential decay** — weight each event by $\exp(-\Delta t / \tau)$. The half-life τ encodes how long an event stays relevant: $\tau=1$ day for fast-moving signals like clicks, $\tau=30$ days for slow-moving taste signals like watch completions.
- **Multiple time windows** — compute the same signal at 1h, 1d, 7d. A user with a high 1h rate and a normal 7d rate is in an unusual session right now; a high 7d with a low 1h has cooled off. Three windows (short/medium/long) cover most cases; more pays compute for diminishing signal.

Production systems frequently ship all three (decay inside each window, smoothing on top) for the handful of rate features that actually matter. Compared to your massive transformer towers, these are tiny.

Pitfalls That Disqualify Candidates

Ok, we've covered some of our sources, the features you can get out of them, and how to encode them for use in the model. What follows are frequently targeted questions from your interviewer with the aim to see how far you've thought the problem. Here are a few common pitfalls to watch out for:

Leakage

An incredibly popular topic for interviews, leakage is when a feature contains information that wouldn't actually be available at serve time. A few examples that all sound innocent:

- Predicting whether a video will be "harmful" using a feature that includes the moderator label assigned after the prediction would be made. Moderator label is the answer.
- Predicting click-through rate using `total_clicks_lifetime` on the impression in question. The training row knows the future.
- Aggregating "user's average click rate" over the entire training set when the user appears in both train and test rows. The test rows leak through the aggregate.

Leaked features make your offline metrics look gorgeous and your online launch flop. So it's always good to ask, for every feature: *could this value actually be known at request time, in production, before the prediction is made?* If the answer requires squinting, the feature is leaky and you at very least want to call out the danger to the interviewer.

Cold start

Some features only exist for entities you have history on. A brand-new user has no `7d_watch_count` and a brand-new video has no `1h_view_velocity`. There's a few common ways to handle this. One is to impute the average value for the feature and use it until you get enough history for something better. This isn't that good because it's misleading: the truth is that we just don't know!

A better solution is to have your model accept a "missing" sentinel feature and decide explicitly what the absence means, or run a different feature set entirely for cold-start cohorts and graduate to the full model once enough behavior accumulates. If you do this, you need to be careful to call out that you'll need to have instances in your training data that are missing (sometimes you can synthetically create them), but by doing so the model can learn the best strategy for handling the missing instances.



Recommendation system questions are quite heavy in cold start discussion and it goes even beyond feature engineering. Often the systems need to not only make sure they're not making mistakes on new items, but also not acting in a way that sabotages new users, items, videos, etc. from getting signal to start their flywheel. Nobody will want to start posting on a platform if they know that only the biggest users are having success.

Feedback loops

Feedback is a really common discussion because it's almost impossible to completely avoid in production systems. In the abstract, this is a self-reinforcing cycle where the model's decisions influence the features that are used to train it. This is a common discussion in recommendation systems, but it can also apply to other domains like fraud detection, harmful content classification, and more. So engagement rates rise on items the model already promotes or reports come in faster on accounts the model already flagged.

Over time the feature stops measuring "is this content engaging" and starts measuring "did the model surface this." Your training data convinces you the model is right.

The fix is rarely to drop the feature (the engagement information is real). It's to add randomized exploration that produces unbiased data, or to log the model's own propensity and adjust the loss using inverse propensity weighting. In an interview, you'll want to name the loop ("aha, there might be a feedback loop here!") and propose mitigation. Senior candidates have invariably spent considerable time investigating and resolving feedback loops in their systems, so they'll appreciate it when you call it out.

Adversarial features

Many systems will have clients or users that try to game the system. This doesn't have to be hackers, this could be instagram influencers trying to get more reach for their posts or upvotes on their Reddit posts. Thinking adversarially will help you identify features that are vulnerable to this.

Some features are robust to adversaries. Some aren't. Content signals (does this look like spam?) are easy for adversaries to fool — the bad guys just generate slightly different content. Temporal and network signals (does this account post in unnatural bursts? is it part of a tightly-clustered subgraph?) are much harder to fake. Our [bot detection breakdown](#) leans on this distinction explicitly.

Cloudflare's writeup on [residential-proxy bot detection](#) is concrete on this: they catch proxied traffic via temporal patterns (peak-hour bursts, inactivity gaps when the host's client closes) and latency anomalies, and not IP reputation, because IP reputation is exactly what the adversary bought their way around. Stripe Radar [scores 1,000+ signals per transaction](#) and leans hard on cross-merchant card-network features — Stripe has seen 90% of cards before, so

a single merchant's fraud signal is dwarfed by the network signal an individual merchant can't replicate.

In adversarial domains, the senior angle is to discuss the *robustness* of each feature group, not just its predictive power. A feature that's 80% accurate today and 30% accurate in six months because adversaries adapted is worth less than a feature that's 70% accurate forever.



If you've ever noticed a particular spam campaign on a social network and wondered why they aren't implementing simple regexes to block them, this is why. As soon as the regexes are in place, the attackers adapt, change their approach, and the spam continues. Worse, the false positives continue long after the real spammers have moved on. Yuck.

Drift

Lastly, drift is a thing in the real world (and doesn't make as much of a showing in academic papers where datasets are static). This means features whose statistical properties change over time even when nothing adversarial is happening. Holiday shopping season shifts the distribution of "average cart value." A new product category shifts the distribution of "category embedding." A system change in upstream logging shifts the units of a numeric feature.

Production teams monitor feature distributions and alert on drift. Meta published in 2024 that they continuously detect feature anomalies at serving time and have automated guardrails that drop or ignore features whose coverage tanks or whose values look corrupted, rather than blindly serving them into the model. Retraining frequently can be a partial solution, but it's not a silver bullet. You might also think "hey I'll just keep multiple years of data and then have features around time, season, whatever" and that also rarely works partly because of sparsity issues (hard to get that volume of data), partly due to retention, and mostly because the model will overfit to the time-based features and not learn the underlying signal.

Being aware of which features are likely to drift and which ones won't is the key to making a system that is reliable not just offline, but online for the days and weeks to come.

Summary

Ok, we've covered a lot here! Features and data are the lifeblood of any AI/ML system, but there's so much to talk about that interviews can get chaotic and messy. The key is to have a structure that you can follow and that the interviewer can follow. You're going to spend **10 minutes or so on data and features in a standard interview**. Here's a reasonable breakdown of how you might spend that time (loose guidelines, don't assume this is a hard rule):

- **~3 minutes** enumerating the sources that matter for the problem, with 2-3 representative signals each. This is where the structure goes on the whiteboard.
- **~2 minutes** on how each source actually gets fed into the model: raw text and images through encoders, sparse IDs through embedding tables, sequences as tokens or aggregates, numeric scalars with log-scaling or bucketing, upstream embeddings imported as-is.
- **~2 minutes** on the one or two normalization tricks the problem actually needs (Bayesian smoothing for low-denominator rates, ratios over raw counts, multiple time windows for drift-prone signals).
- **~2 minutes** on the pitfall most likely to bite for this problem: leakage if you're predicting the future, train/serve skew if your features are slow-moving, cold start if you've got new entities, adversarial robustness if you're in trust and safety.
- **A minute of buffer** for the earmarks you've left open. The interviewer either pulls you back to one or you move on.

With that in mind, here's roughly what the opening of a senior feature discussion might sound like:

"For this problem the heavy hitters are item content (the post text and image), the actor (the author's history), and behavioral signals on the post itself (recent reactions, share velocity). Network signals matter less unless we're worried about coordinated inauthentic behavior, which I'll flag and come back to. Let's start with the content features ..."

Then you spend time filling out the structure you created. When you hit a normalization-sensitive signal like `negatives_per_view`, you flag it and propose Bayesian smoothing without making the interviewer ask. When you hit something with train/serve skew risk, you propose a feature store. When you hit a leakage risk, you flag it. And so on.



The most common feature-discussion failure is not knowing when to stop. **Earmark depth, don't burn time on it.** "There's a lot more I'd want to discuss on behavioral signals

but I want to keep moving. Let me know if you want me to come back" is one of the most powerful sentences in the ML system design interview. It signals depth without spending the time to demonstrate it, and the interviewer will pull you back if they actually want it.

When the interviewer pushes on a specific source, you go deeper. When they don't, you keep moving. Since there's so much to discuss, your interviewer wants to see a bit of breadth (are you thinking about the problem holistically) and a bit of depth (do you understand the nuances of the problem and the implications of your choices). So don't lose your mind if you feel like you're not covering everything, you often can't!